

OmniLitter

Sistema multiplataforma para la gestión y análisis de datos de
contaminación por residuos



Escuela Técnica Superior de
Ingeniería Informática

Trabajo Fin de Grado

Grado: Ingeniería Informática - Ingeniería del Software

Centro: ETSII — Universidad de Sevilla

Curso académico: 2025/26

Autores:

- Alejandro Castilla — alecasbar@alum.us.es
- Ignacio Martínez Díaz — ignmardial@alum.us.es

Tutor académico: Cruz Mata, Fermín

Cotutor: Ortega Rodríguez, Francisco Javier

Cliente / entidad colaboradora: Daniel González Fernández /
Universidad de Cádiz

Índice

1. Introducción	1
2. Descripción del Proyecto	2
2.1. Objetivos	2
3. Arquitectura del Sistema	3
3.1. Visión general	3
4. Tecnologías de Desarrollo	4
4.1. Criterios de selección	4
4.2. Tecnologías empleadas en los mockups	4
4.3. Portal web	5
4.4. Aplicación móvil	5
4.5. Backend y persistencia	6
4.6. Justificación de SQLite frente a WatermelonDB	7
5. Planificación y Gestión	8
5.1. Metodología de trabajo	8
5.2. Gestión de reuniones	8
5.3. Gestión del alcance	8
6. Requisitos	10
6.1. Proceso de toma de requisitos	10
6.2. Reunión inicial con el cliente	10
6.3. Reunión de validación de mockups	11
6.4. Requisitos funcionales principales	11
6.5. Requisitos no funcionales	12
6.6. Requisitos de datos científicos	13
6.7. Trazabilidad entre reuniones y requisitos	13
7. Diseño de la Solución	14
7.1. Visión General de la Arquitectura	14
7.2. Diseño de la Base de Datos	15
7.3. Diseño de la API REST	19
7.4. Diseño del Portal Web	21
7.5. Diseño de la Aplicación Móvil	22
7.6. Mecanismo de Sincronización Offline	23
A. Declaración del Uso de IA	26
A.1. Usos realizados	26
A.2. Ejemplos de prompts utilizados	27
A.3. Autoría y revisión crítica	27

B. Requisitos Detallados y Validación con Cliente	28
B.1. Resumen de reuniones con el cliente	28
B.2. Requisitos obtenidos en la reunión inicial	28
B.3. Validación de requisitos mediante mockups	29
B.4. Requisitos funcionales detallados	29
B.5. Requisitos no funcionales detallados	30
B.6. Requisitos de datos científicos	30

Índice de cuadros

1.	Tecnologías empleadas en los mockups funcionales.	5
2.	Stack previsto para el portal web.	5
3.	Stack previsto para la aplicación móvil.	6
4.	Stack previsto para backend y persistencia.	6
5.	Resumen de requisitos funcionales principales.	12
6.	Trazabilidad entre reuniones con el cliente y requisitos.	13
7.	Descripción de entidades del modelo de datos.	16
8.	Principales <i>endpoints</i> de la API REST de OmniLitter.	19
9.	Estados de sincronización y su significado.	24
10.	Detalle del uso de IA en el trabajo.	26
11.	Resumen de reuniones mantenidas con el cliente.	28

Índice de figuras

1. Arquitectura general propuesta para OmniLitter. 14
2. Diagrama entidad-relación del sistema OmniLitter. 16

1. Introducción

Llevar la cuenta de todo lo que uno ve se ha convertido en algo

2. Descripción del Proyecto

2.1. Objetivos

3. Arquitectura del Sistema

3.1. Visión general

4. Tecnologías de Desarrollo

En este capítulo se describen las tecnologías seleccionadas para el desarrollo de OmniLitter y la justificación de su elección. La selección se ha realizado teniendo en cuenta tres restricciones principales: el alcance de un Trabajo de Fin de Grado desarrollado por dos personas, la necesidad de disponer de una aplicación móvil capaz de trabajar sin conexión y la conveniencia de mantener un stack homogéneo basado en TypeScript.

4.1. Criterios de selección

Los criterios utilizados para seleccionar las tecnologías han sido los siguientes:

- **Homogeneidad tecnológica:** se prioriza el uso de TypeScript tanto en el frontend como en el backend para reducir el cambio de contexto durante el desarrollo.
- **Capacidad offline:** la aplicación móvil debe poder crear sesiones, observaciones y fotografías sin depender de la conexión a internet.
- **Madurez del ecosistema:** se eligen tecnologías ampliamente usadas, con documentación suficiente y comunidad activa.
- **Facilidad de despliegue:** el sistema debe poder desplegarse en servicios accesibles para un proyecto académico, como Vercel, Render, Railway o una máquina virtual sencilla.
- **Escalabilidad razonable:** aunque el proyecto no persigue una escala industrial, el diseño debe permitir incorporar nuevos grupos, campañas y datos de campo sin rehacer la arquitectura.

4.2. Tecnologías empleadas en los mockups

Durante la fase inicial se han construido dos prototipos funcionales con datos en memoria: un portal web y una simulación de aplicación móvil en navegador. Ambos prototipos permiten validar flujos de usuario con el cliente antes de desarrollar la versión final.

Cuadro 1: Tecnologías empleadas en los mockups funcionales.

Tecnología	Uso en el proyecto
React 18	Construcción de interfaces de usuario para el portal web y el mockup móvil.
TypeScript	Tipado estático de componentes, rutas, estado y datos de ejemplo.
Vite	Servidor de desarrollo y empaquetado rápido de los prototipos.
Tailwind CSS	Sistema de estilos utilitario para construir pantallas responsive.
React Router	Definición de rutas públicas, rutas protegidas y navegación entre pantallas.
Recharts	Gráficas y visualizaciones estadísticas del dashboard del portal web.
Lucide React / Material Icons	Iconografía de la interfaz.

4.3. Portal web

El portal web definitivo se plantea como una aplicación *Single Page Application* desarrollada con React, TypeScript y Vite. Su responsabilidad principal será permitir a investigadores y administradores consultar observaciones, gestionar usuarios y grupos, visualizar mapas y exportar datos.

Cuadro 2: Stack previsto para el portal web.

Tecnología	Responsabilidad
React	Construcción de componentes reutilizables y pantallas del portal.
TypeScript	Definición de tipos de dominio, respuestas de API y props de componentes.
Vite	Compilación, servidor de desarrollo y generación del bundle final.
Tailwind CSS	Sistema visual responsive y consistente con los mockups.
React Router	Gestión de rutas públicas y protegidas.
Axios / Fetch API	Comunicación con la API REST del backend.
Leaflet / React Leaflet	Visualización geográfica de observaciones en mapas interactivos.
Recharts	Dashboards, rankings y tendencias temporales.

4.4. Aplicación móvil

La aplicación móvil se plantea con React Native y Expo, utilizando SQLite mediante el paquete `expo-sqlite` como base de datos local. El objetivo es que el usuario pueda

completar una jornada de muestreo aunque el dispositivo no tenga conexión a internet.

Cuadro 3: Stack previsto para la aplicación móvil.

Tecnología	Responsabilidad
React Native	Desarrollo multiplataforma para Android e iOS.
Expo	Toolchain de desarrollo, compilación y acceso a módulos nativos.
TypeScript	Tipado de pantallas, servicios, modelos locales y respuestas de API.
expo-sqlite / SQLite	Persistencia local de sesiones, observaciones, fotografías pendientes y cola de sincronización.
expo-location	Captura de coordenadas GPS asociadas a sesiones y observaciones.
expo-camera	Captura de fotografías y obtención de metadatos cuando sea posible.
expo-task-manager / expo-background-task	Ejecución de tareas de sincronización en segundo plano cuando el sistema operativo lo permita.
React Navigation	Navegación entre pantallas principales y flujos de captura.

4.5. Backend y persistencia

El backend centralizará la lógica de negocio, la autenticación, el control de acceso por grupo y la sincronización de datos procedentes de dispositivos móviles. Se plantea como una API REST desarrollada con Node.js, Express y TypeScript.

Cuadro 4: Stack previsto para backend y persistencia.

Tecnología	Responsabilidad
Node.js	Entorno de ejecución del backend.
Express	Definición de rutas, middlewares y controladores REST.
TypeScript	Tipado de servicios, DTOs, entidades y errores.
PostgreSQL	Base de datos relacional principal del sistema.
Prisma	ORM para modelar entidades, migraciones y acceso a datos.
JWT	Autenticación mediante tokens de acceso y refresco.
bcrypt	Almacenamiento seguro de contraseñas mediante hash.
multer / almacenamiento de ficheros	Recepción y almacenamiento de fotografías asociadas a observaciones.

4.6. Justificación de SQLite frente a WatermelonDB

Aunque inicialmente se valoró WatermelonDB como solución local *offline-first*, se ha decidido emplear SQLite directamente para ajustar mejor el alcance del TFG. SQLite permite cubrir los requisitos principales de la aplicación móvil: almacenar sesiones, observaciones, fotografías pendientes y una cola de sincronización local.

La decisión reduce la complejidad técnica, evita depender de una capa adicional de sincronización y facilita que los autores controlen de forma explícita el esquema local y el flujo de subida de datos. El patrón *offline-first* se mantiene, pero se implementa mediante una tabla local de cola de sincronización y servicios propios de lectura/escritura sobre SQLite.

5. Planificación y Gestión

El desarrollo de OmniLitter se ha planteado de forma iterativa, combinando tareas de análisis, diseño, prototipado e implementación. Esta aproximación resulta adecuada para un proyecto con cliente real, ya que permite validar decisiones antes de invertir esfuerzo en una implementación definitiva.

5.1. Metodología de trabajo

La metodología seguida se basa en ciclos cortos de trabajo:

1. Recogida de necesidades con el cliente y tutores.
2. Definición de requisitos funcionales, no funcionales y de datos científicos.
3. Elaboración de mockups funcionales para validar flujos de usuario.
4. Revisión con el cliente y refinamiento del prototipo.
5. Diseño de la arquitectura, base de datos y componentes del sistema.
6. Implementación progresiva de las partes finales del proyecto.

Aunque no se ha aplicado una metodología ágil formal completa, el proyecto adopta una forma de trabajo incremental: cada avance genera un artefacto revisable (documento de requisitos, mockup, diagrama, módulo de código o capítulo de memoria) que se contrasta antes de continuar.

5.2. Gestión de reuniones

Las reuniones con el cliente han tenido un papel central en la definición del alcance. La primera reunión permitió identificar el problema y las funcionalidades esperadas. Posteriormente, una segunda reunión sirvió para revisar los mockups, confirmar la cobertura de los flujos principales y reforzar prioridades como el funcionamiento sin conexión, la separación por grupos y la trazabilidad científica.

Estas reuniones se documentan en el Capítulo 6 y en el Anexo B, donde se relacionan las necesidades detectadas con los requisitos del sistema.

5.3. Gestión del alcance

El alcance se ha dividido en dos niveles:

- **Alcance funcional validado mediante mockups:** portal web y aplicación móvil con flujos navegables, datos de ejemplo, roles simulados y pantallas principales implementadas.

- **Alcance de implementación final:** backend REST, base de datos, autenticación, sincronización offline, persistencia de observaciones y conexión progresiva de los clientes al backend.

Esta separación permite demostrar primero la experiencia de uso y después construir la infraestructura técnica necesaria para convertir el prototipo en una aplicación real.

6. Requisitos

Este capítulo recoge el proceso seguido para obtener, contrastar y refinar los requisitos de OmniLitter. Dado que el proyecto parte de una necesidad real planteada por un cliente experto en el dominio, la definición de requisitos no se limitó a una fase inicial cerrada, sino que evolucionó a partir de reuniones de toma de requisitos, validación de mockups y revisión de los flujos propuestos.

6.1. Proceso de toma de requisitos

La toma de requisitos se realizó en dos fases principales:

1. **Reunión inicial de toma de requisitos.** En esta primera reunión se presentó el problema: renovar y ampliar una herramienta existente de monitorización de residuos, incorporando nuevos entornos de muestreo, gestión de usuarios y grupos, funcionamiento sin conexión y un portal web de consulta.
2. **Reunión de contraste con mockups.** Tras construir los primeros prototipos interactivos del portal web y de la aplicación móvil, se revisó con el cliente si los flujos propuestos cubrían los requisitos inicialmente identificados. Esta reunión sirvió para validar el enfoque general y detectar qué partes debían priorizarse o explicarse con mayor detalle en el diseño.

Esta forma de trabajo permitió reducir la incertidumbre del proyecto: primero se recogieron las necesidades de alto nivel y posteriormente se contrastaron con pantallas navegables antes de abordar la implementación definitiva.

6.2. Reunión inicial con el cliente

En la reunión inicial se identificaron las necesidades fundamentales del sistema. El cliente planteó que la nueva solución debía superar el alcance de la aplicación existente, centrada principalmente en residuos flotantes, para permitir la captura de datos en distintos entornos naturales.

Los puntos principales extraídos de esta reunión fueron:

- El sistema debe estar formado por una **aplicación móvil** para trabajo de campo y un **portal web** para consulta, gestión y análisis.
- La aplicación móvil debe permitir registrar observaciones de residuos en entornos acuáticos y terrestres, incluyendo ríos, mares, playas y bosques.
- Cada observación debe guardar, siempre que sea posible, posición GPS, fecha, hora, tipo de residuo, cantidad y fotografía.

- La aplicación debe funcionar sin conexión, ya que el trabajo de campo puede realizarse en zonas con cobertura limitada.
- El sistema debe permitir trabajar con grupos de investigación y roles de usuario, evitando que los datos de un grupo sean visibles por otros grupos.
- El portal web debe permitir visualizar los datos mediante listados, mapas y métricas agregadas.
- La taxonomía de residuos debe estar organizada de forma navegable y ser suficientemente flexible para ampliarse en el futuro.

6.3. Reunión de validación de mockups

Una vez generados los mockups funcionales, se realizó una segunda reunión para contrastar con el cliente si los flujos planteados respondían correctamente a sus necesidades. En esta revisión se comprobó que los prototipos permitían representar los procesos principales del sistema: autenticación, selección de grupo, creación de sesiones de muestreo, registro de observaciones, consulta de mapas y visualización de datos desde el portal.

La reunión no introdujo una lista cerrada de nuevos requisitos funcionales, pero sí sirvió para confirmar la importancia de tres aspectos que debían mantenerse en el diseño final:

- La aplicación móvil debía seguir siendo prioritaria para el trabajo de campo y conservar el enfoque *offline-first*.
- El portal web debía mantener una separación clara entre datos de distintos grupos de investigación.
- La captura de datos debía preservar metadatos científicos suficientes para que las observaciones pudieran analizarse posteriormente.

6.4. Requisitos funcionales principales

Los requisitos funcionales definen el comportamiento esperado del sistema. A continuación se resumen los más relevantes para el alcance del proyecto:

Cuadro 5: Resumen de requisitos funcionales principales.

ID	Descripción	Prioridad
RF-001	Inicio de sesión en portal web y aplicación móvil.	Crítica
RF-002	Cierre de sesión y revocación de sesión local.	Alta
RF-004	Soporte de roles y control de acceso por grupo.	Crítica
RF-005	Creación de grupos y asociación de usuarios a grupos.	Crítica
RF-006	Selección de grupo activo para filtrar datos y acciones.	Alta
RF-007	Trabajo por sesiones de muestreo en la aplicación móvil.	Crítica
RF-008	Captura de coordenadas GPS al iniciar sesiones de muestreo y observaciones.	Crítica
RF-010	Adjuntar fotografías a una observación.	Alta
RF-012	Navegación por taxonomía jerárquica de residuos.	Crítica
RF-015	Registro de cantidad observada como entero positivo.	Crítica
RF-020	Revisión de sesión antes de finalizarla.	Alta
RF-022	Funcionamiento sin conexión durante la captura de campo.	Crítica
RF-023	Cola de sincronización y subida automática al recuperar conectividad.	Crítica
RF-027	Listado de sesiones y observaciones con filtros en el portal.	Alta
RF-028	Visualización de observaciones en mapa.	Alta
RF-030	Dashboard con métricas agregadas.	Media
RF-032	Gestión de usuarios desde el portal.	Alta
RF-033	Gestión de grupos y membresías desde el portal.	Alta
RF-035	Soporte de interfaz en español e inglés.	Media

6.5. Requisitos no funcionales

Los requisitos no funcionales recogen restricciones de calidad, seguridad y operación. Los más relevantes son:

- **Seguridad:** toda la comunicación entre clientes y backend debe realizarse mediante HTTPS/TLS.
- **Aislamiento de datos:** los usuarios solo podrán acceder a datos de los grupos a los que pertenecen.
- **Offline-first:** las operaciones de captura en campo no deben depender de conectividad.
- **Fiabilidad de sincronización:** los datos no deben perderse ante cortes de red o cierres inesperados de la aplicación.
- **Usabilidad:** la aplicación móvil debe estar optimizada para uso en campo, con botones claros, formularios rápidos y navegación simple.
- **Trazabilidad:** el sistema debe mantener logs de operaciones relevantes y relacionar requisitos con pruebas.

6.6. Requisitos de datos científicos

Al tratarse de una herramienta para captura y análisis de datos científicos, el sistema debe preservar suficiente información contextual para que los datos sean interpretables y reproducibles. En particular:

- La taxonomía debe presentarse de forma jerárquica sin perder el código identificador de cada tipo de residuo.
- Las sesiones deben almacenar metadatos de muestreo como entorno, ancho de transecto, duración, protocolo, versión de aplicación y versión de taxonomía.
- Las fotografías deben conservar metadatos relevantes cuando estén disponibles, como fecha de captura, dimensiones o coordenadas EXIF.
- Los datos exportados deben incluir campos suficientes para análisis posteriores y, cuando sea posible, un diccionario de columnas y unidades.
- La versión de taxonomía utilizada debe quedar asociada a las sesiones y observaciones para garantizar reproducibilidad.

6.7. Trazabilidad entre reuniones y requisitos

La tabla 6 resume la relación entre las reuniones con el cliente y los requisitos derivados.

Cuadro 6: Trazabilidad entre reuniones con el cliente y requisitos.

Fuente		Necesidad identificada	Requisitos relacionados
Reunión inicial		Aplicación móvil de campo y portal web de gestión.	RF-001, RF-027, RF-030
Reunión inicial		Captura de observaciones con GPS, fecha, tipo de residuo, cantidad y fotografía.	RF-008, RF-010, RF-012, RF-015
Reunión inicial		Funcionamiento sin conexión y sincronización posterior.	RF-022, RF-023, fiabilidad de sincronización
Reunión inicial		Gestión de grupos, usuarios y privacidad de datos.	RF-004, RF-005, RF-006, RF-032, RF-033, aislamiento de datos
Validación de mockups	de	Confirmación del enfoque <i>offline-first</i> y de la captura por sesiones.	RF-007, RF-020, RF-022, RF-023
Validación de mockups	de	Confirmación de la separación por grupos y la trazabilidad científica.	RF-004, RF-006, requisitos de datos científicos

El detalle ampliado de requisitos y de su validación con mockups se incluye en el Anexo B.

7. Diseño de la Solución

Este capítulo describe las decisiones de diseño adoptadas para materializar los requisitos funcionales y no funcionales de OmniLitter en una arquitectura concreta. Se abordan, en orden de abstracción descendente, la arquitectura general del sistema, el diseño de la base de datos, el contrato de la API REST, la estructura de componentes del portal web y de la aplicación móvil, y el mecanismo de sincronización *offline*.

7.1. Visión General de la Arquitectura

OmniLitter sigue una arquitectura de **tres capas** clásica adaptada a un contexto multi-plataforma con requisito de operación sin conexión:

1. **Capa de presentación.** Comprende dos clientes independientes: el portal web, orientado a la consulta y gestión de datos por parte de investigadores y administradores, y la aplicación móvil, orientada a la captura de observaciones en campo.
2. **Capa de lógica de negocio.** Una API REST centralizada planteada con Node.js + Express actúa como único punto de acceso a los datos, aplica las reglas de negocio y gestiona la autenticación mediante *tokens* JWT.
3. **Capa de persistencia.** PostgreSQL almacena el estado canónico del sistema. Adicionalmente, la aplicación móvil mantiene una base de datos local SQLite que permite operar sin conexión y sincronizarse en diferido.

La figura 1 muestra un diagrama de la arquitectura del sistema.

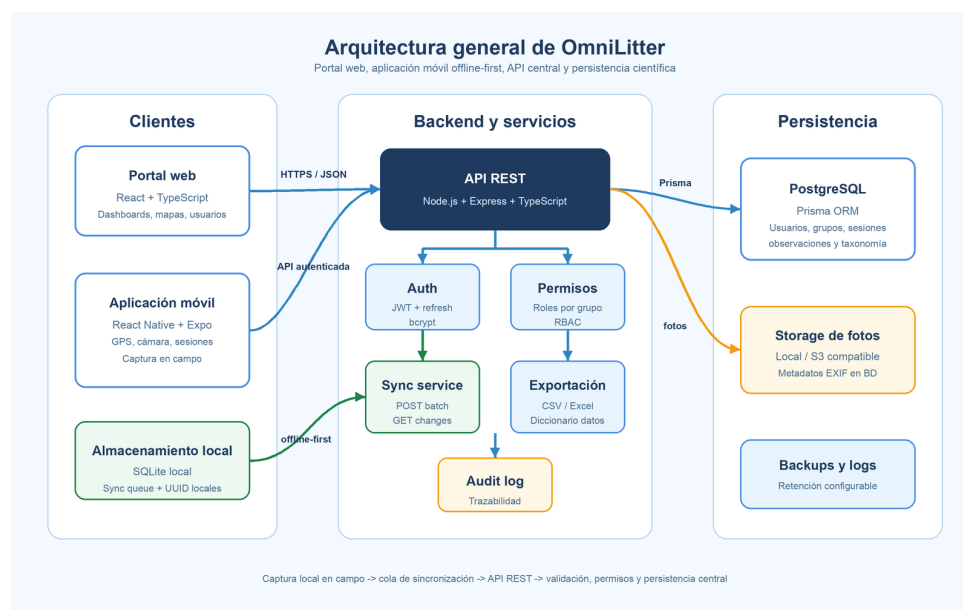


Figura 1: Arquitectura general propuesta para OmniLitter.

7.1.1. Justificación de las decisiones arquitectónicas principales

API REST como único punto de acceso. Centralizar la lógica de negocio en la API evita duplicidades entre los dos clientes y facilita la incorporación de nuevos consumidores en el futuro (p.ej., exportaciones automatizadas por *scripts* externos o integraciones con repositorios científicos).

Separación entre portal web y aplicación móvil. Aunque comparten la misma API, los dos clientes tienen requisitos de experiencia de usuario muy distintos: el portal prioriza la visualización analítica y la gestión, mientras que la app móvil prioriza la captura rápida en condiciones de campo adversas (luz solar, guantes, sin conexión). Mantenerlos como proyectos independientes permite evolucionar cada uno a su propio ritmo.

Base de datos local en el móvil. El trabajo de campo se realiza habitualmente en zonas costeras, ribereñas o forestales con cobertura de red intermitente o nula. La decisión de embeber una base de datos local en el dispositivo es, por tanto, un requisito no negociable para la viabilidad del producto.

7.2. Diseño de la Base de Datos

7.2.1. Modelo relacional

El modelo de datos de OmniLitter se compone de diez entidades que se agrupan en tres áreas funcionales:

- **Gestión de acceso:** USER, GROUP, MEMBERSHIP.
- **Organización científica:** CAMPAIGN, SURVEY_SESSION.
- **Datos de campo:** OBSERVATION, PHOTO, TAXONOMY_ITEM, FAVORITE, AUDIT_LOG.

La figura 2 muestra el diagrama entidad-relación del sistema.

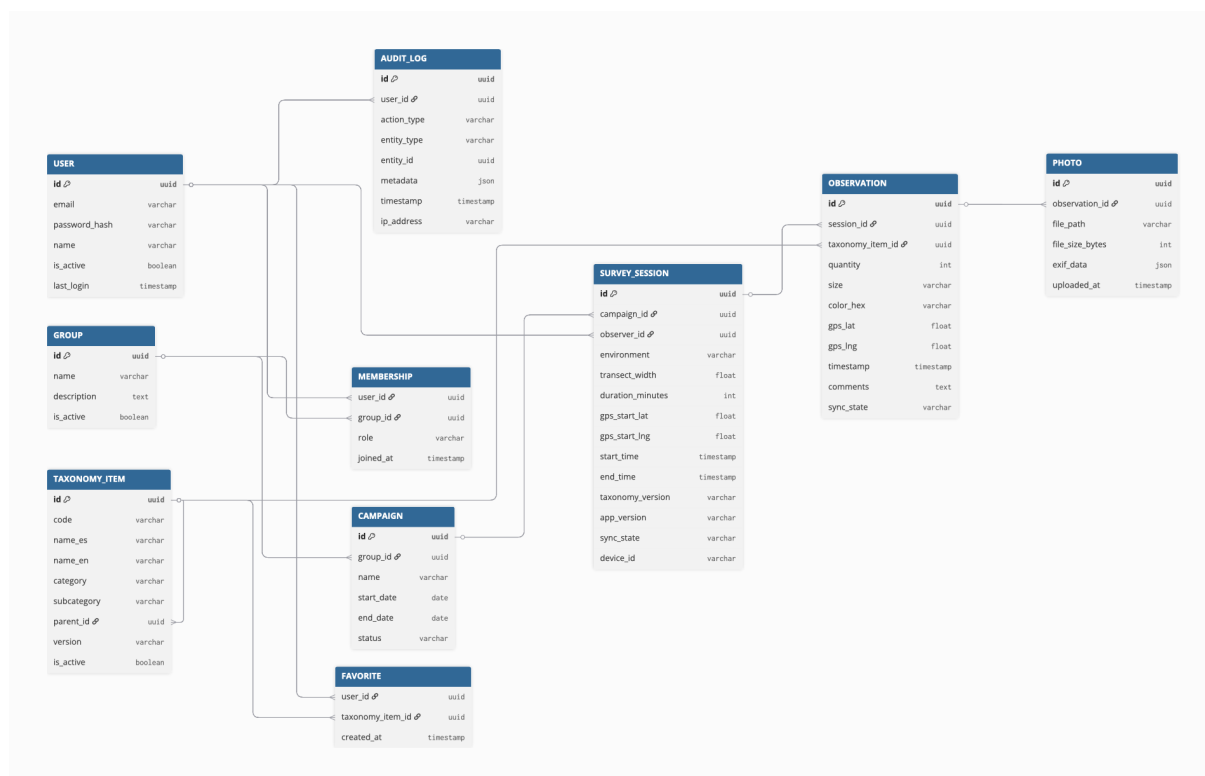


Figura 2: Diagrama entidad-relación del sistema OmniLitter.

7.2.2. Descripción de entidades

La tabla 7 describe los atributos clave de cada entidad y su propósito en el sistema.

Cuadro 7: Descripción de entidades del modelo de datos.

Entidad	Atributos principales	Descripción
USER	id (UUID), email, password_hash, name, is_active, last_login	Representa a cada persona registrada en el sistema. Las contraseñas se almacenan como <i>hash</i> bcrypt y nunca en texto plano.
GROUP	id (UUID), name, description, is_active	Equipo de investigación. Un grupo puede ser archivado sin eliminar sus datos históricos, preservando la trazabilidad científica.
MEMBERSHIP	user_id (FK), group_id (FK), role, joined_at	Tabla de unión que asocia usuarios con grupos asignándoles un rol. Un mismo usuario puede tener roles distintos en grupos distintos.

Entidad	Atributos principales	Descripción
CAMPAIGN	id (UUID), group_id (FK), name, start_date, end_date, status	Proyecto científico de muestreo. Agrupa varias sesiones de campo bajo un objetivo común (p.ej., «Monitoreo anual Playa de la Caleta 2026»).
SURVEY_SESSION	id (UUID), campaign_id (FK), observer_id (FK), environment, transect_width, duration_minutes, gps_start_lat/lng, start_time, end_time, taxonomy_version, app_version, sync_state, device_id	Sesión de muestreo individual en campo. Registra todos los metadatos científicos necesarios para la reproducibilidad: protocolo, versión de taxonomía utilizada y versión de la aplicación. El campo sync_state gestiona el ciclo de vida de la sincronización.
OBSERVATION	id (UUID), session_id (FK), taxonomy_item_id (FK), quantity, size, color_hex, gps_lat/lng, timestamp, taxonomy_version, comments, sync_state	Registro de un ítem de residuo observado. Constituye la unidad mínima de dato científico del sistema.
PHOTO	id (UUID), observation_id (FK), file_path, file_size_bytes, exif_data (JSONB), uploaded_at	Imagen asociada a una observación. Los metadatos EXIF (coordenadas GPS embebidas en la imagen, fecha de captura, modelo de cámara) se almacenan en formato JSONB para máxima flexibilidad.
TAXONOMY_ITEM	id (UUID), code, name_es, name_en, category, subcategory, parent_id (FK self-ref), version, is_active	Catálogo de tipos de residuo basado en la <i>Joint List of Litter Categories</i> . La estructura jerárquica auto-referenciada (categoría → subcategoría → ítem) permite navegar la taxonomía sin conexión, ya que se descarga completa al iniciar sesión.

Entidad	Atributos principales	Descripción
FAVORITE	<code>user_id</code> (FK), <code>taxonomy_item_id</code> (FK), <code>created_at</code>	Ítems de taxonomía marcados como favoritos por el usuario para agilizar la captura de observaciones en campo.
AUDIT_LOG	<code>id</code> , <code>user_id</code> (FK), <code>action_type</code> , <code>entity_type</code> , <code>entity_id</code> , <code>metadata</code> (JSONB), <code>timestamp</code> , <code>ip_address</code>	Registro de auditoría que almacena las acciones relevantes sobre el sistema (inicio de sesión, creación y edición de observaciones, exportaciones de datos, cambios de configuración). Garantiza la trazabilidad exigida en entornos de investigación científica.

7.2.3. Decisiones de diseño de la base de datos

UUID como clave primaria en registros sincronizables. Los registros que pueden crearse en el dispositivo móvil antes de comunicarse con el servidor utilizan UUID v4 generados en el cliente. Esto resuelve el problema de colisiones de claves en escenarios de creación offline: dos dispositivos diferentes que creen una observación sin conexión obtendrán identificadores globalmente únicos que no entrarán en conflicto cuando se sincronicen con el servidor.

Rol en MEMBERSHIP, no en USER. El rol de un investigador no es una propiedad intrínseca de la persona, sino de su relación con un grupo concreto. Un mismo investigador puede ser administrador en su grupo principal y colaborador invitado en otro grupo. Modelar el rol en la tabla de unión es, por tanto, la decisión correcta desde el punto de vista semántico.

Versión de taxonomía en sesiones y observaciones. La taxonomía de residuos evoluciona con el tiempo a medida que la comunidad científica añade o reorganiza categorías. Registrar la versión exacta utilizada en cada sesión permite reconstruir con exactitud qué definición se aplicó a cada observación, lo que es un requisito básico de reproducibilidad científica.

JSONB para metadatos heterogéneos. Los campos `exif_data` y `metadata` utilizan el tipo JSONB de PostgreSQL para almacenar datos cuya estructura puede variar (distintos modelos de cámara incluyen campos EXIF diferentes, distintas acciones de auditoría tienen distintos contextos). Esta decisión evita la proliferación de columnas opcionales con valor nulo y preserva la flexibilidad para añadir nuevos atributos sin migrar el esquema.

7.3. Diseño de la API REST

La API sigue los principios REST: interfaz uniforme, comunicación sin estado, recursos identificados por URI y representaciones en formato JSON. Todos los *endpoints* protegidos requieren el encabezado `Authorization: Bearer <token>`, donde el *token* es un JWT firmado emitido en el *endpoint* de autenticación.

La tabla 8 recoge los *endpoints* principales agrupados por recurso.

Cuadro 8: Principales *endpoints* de la API REST de OmniLitter.

Método	Ruta	Protegido	Descripción
<i>Autenticación</i>			
POST	/api/auth/login	No	Emite JWT de acceso y refresco
POST	/api/auth/refresh	No	Renueva el JWT de acceso
POST	/api/auth/logout	Sí	Invalida el <i>token</i> de refresco
<i>Usuarios</i>			
GET	/api/users	Sí (admin)	Lista todos los usuarios
POST	/api/users	Sí (admin)	Crea un nuevo usuario
GET	/api/users/:id	Sí	Obtiene un usuario
PATCH	/api/users/:id	Sí (admin)	Actualiza datos o estado
<i>Grupos</i>			
GET	/api/groups	Sí	Lista grupos del usuario autenticado
POST	/api/groups	Sí (admin)	Crea un grupo
PATCH	/api/groups/:id	Sí (admin)	Edita nombre/descripción
DELETE	/api/groups/:id/archive	Sí (admin)	Archiva un grupo
POST	/api/groups/:id/members	Sí (admin)	Añade miembro
DELETE	/api/groups/:id/members/:uid	Sí (admin)	Elimina miembro
<i>Sesiones de muestreo</i>			
GET	/api/sessions	Sí	Lista sesiones accesibles
POST	/api/sessions	Sí	Crea una sesión
GET	/api/sessions/:id	Sí	Obtiene una sesión con observaciones

Método	Ruta	Protegido	Descripción
PATCH	/api/sessions/:id	Sí	Actualiza metadatos
<i>Observaciones</i>			
GET	/api/sessions/:id/observations	Sí	Lista observaciones de sesión
POST	/api/sessions/:id/observations	Sí	Registra una observación
GET	/api/observations	Sí	Lista todas (filtros opcionales)
<i>Fotos</i>			
POST	/api/observations/:id/photos	Sí	Sube una foto (multipart)
GET	/api/photos/:id	Sí	Sirve el archivo de imagen
<i>Taxonomía</i>			
GET	/api/taxonomy	Sí	Descarga la taxonomía completa
GET	/api/taxonomy?version=:v	Sí	Descarga versión específica
<i>Sincronización</i>			
POST	/api/sync/batch	Sí	Sube cola de registros pendientes
GET	/api/sync/changes	Sí	Descarga cambios desde last_sync
<i>Exportación</i>			
GET	/api/export/csv	Sí (admin)	Exporta observaciones en CSV científico

7.3.1. Autenticación y autorización

El sistema se diseña con autenticación basada en JWT y dos *tokens*: un **token de acceso** de corta duración (15 minutos) y un **token de refresco** de larga duración (7 días). El token de acceso se envía en cada petición en el encabezado **Authorization**; el de refresco se utiliza exclusivamente para solicitar un nuevo token de acceso cuando el anterior expira.

La autorización sigue un modelo de control de acceso basado en roles (RBAC) a nivel de grupo. El *middleware* de autorización verifica, para cada petición, que el usuario autenticado posee el rol requerido en el grupo al que pertenece el recurso solicitado. Esto permite que una misma persona sea administradora en un grupo y colaboradora en otro,

con permisos diferentes en cada contexto.

7.4. Diseño del Portal Web

7.4.1. Arquitectura de componentes

El portal web se diseña como una *Single Page Application* (SPA) con React 18 y TypeScript, empaquetada con Vite y estilizada con Tailwind CSS. La navegación entre páginas es gestionada por React Router, que define dos niveles de rutas:

- **Rutas públicas:** la página de inicio (/) y el formulario de inicio de sesión (/login), accesibles sin autenticación.
- **Rutas protegidas:** todas las páginas bajo /app/*, protegidas por un componente `PrivateRoute` que verifica el estado de autenticación y redirige al *login* si no existe sesión activa.

La estructura de componentes sigue una jerarquía de tres niveles: el `Layout` general (barra lateral, encabezado, selector de grupo activo) envuelve las páginas individuales, que a su vez utilizan componentes reutilizables (tarjetas de estadística, tablas filtrables, modales de confirmación).

7.4.2. Gestión de estado

En el prototipo del portal web, `localStorage` se utiliza como mecanismo de persistencia de la sesión del usuario entre recargas de página, almacenando el estado necesario para simular autenticación, rol y grupo activo. En la implementación final, este comportamiento deberá integrarse con los *tokens* emitidos por el backend y con una estrategia de refresco de sesión.

En el mockup actual, la comunicación entre el selector de grupo activo (ubicado en la barra lateral) y el componente Dashboard se resuelve mediante un evento personalizado del DOM (`omnilitter:groupChanged`). Esta solución es suficiente para el prototipo y podrá sustituirse por un contexto global o una librería de estado si la implementación final aumenta de complejidad.

7.4.3. Flujo de navegación y control de acceso por rol

Las secciones de **Grupos** y **Usuarios** son exclusivas del rol de administrador y no aparecen en la barra de navegación para los colaboradores. El Dashboard ajusta el contenido mostrado según el rol: los administradores ven estadísticas globales del grupo activo, mientras que los colaboradores ven únicamente sus propias métricas. La tabla de obser-

vaciones filtra los registros por autor para los colaboradores y muestra todos para los administradores.

7.5. Diseño de la Aplicación Móvil

7.5.1. Stack tecnológico y estructura de pantallas

La aplicación móvil está planteada con **React Native** y **Expo**, usando *development builds* o *prebuild* cuando sea necesario acceder a capacidades nativas del dispositivo. Esta aproximación permite mantener las ventajas del ecosistema Expo sin renunciar a módulos como `expo-location`, `expo-camera` y las tareas de sincronización en segundo plano mediante `expo-task-manager` y `expo-background-task`.

La navegación está organizada en dos niveles:

- **Navegación de primer nivel (tab bar):** cinco pestañas inferiores (Inicio, Sesiones, Mapa, Sincronización, Perfil), visibles únicamente en las pantallas principales.
- **Navegación de pila (*stack*):** flujos de varios pasos como la creación de una nueva sesión o la captura de una observación, que se presentan sin el *tab bar*.

7.5.2. Gestión de estado global

El estado global de la aplicación se gestiona mediante el patrón **Context + useReducer** de React, sin dependencias externas de gestión de estado. El contexto `AppContext` expone el estado completo (sesiones activas, tipos de residuo, datos del usuario, estado de sincronización) y un *dispatcher* con acciones tipadas en TypeScript. Este patrón proporciona la predictibilidad de Redux con una superficie de código significativamente menor, adecuada para la escala del proyecto.

7.5.3. Base de datos local con SQLite

SQLite, integrado en Expo mediante `expo-sqlite`, actúa como base de datos local de la aplicación móvil. Se elige por ser una solución madura, ligera y suficientemente potente para almacenar el volumen de datos esperado durante una jornada de muestreo. Además, permite mantener control directo sobre el esquema local y sobre la cola de sincronización, reduciendo la complejidad respecto a incorporar una capa adicional de abstracción.

El esquema local replica el subconjunto del modelo de datos necesario para la operación en campo: `survey_sessions`, `observations`, `photos` y `sync_queue`, además de la taxonomía completa cargada al iniciar sesión. La tabla `sync_queue` registra las operaciones pendientes de subida al servidor junto con su *payload*, número de intentos y estado de sincronización.

7.5.4. Flujo de captura de una observación

El flujo principal de la aplicación sigue esta secuencia:

1. El investigador inicia una nueva sesión de muestreo, especificando el nombre, el tipo de entorno y los parámetros del transecto.
2. Para cada ítem encontrado, abre la pantalla de nueva observación, selecciona el tipo de residuo a través del modal de taxonomía (organizado en cuadrícula visual por categorías e iconos), e introduce cantidad, tamaño, color y comentarios.
3. Opcionalmente captura una foto geolocalizada del residuo.
4. Al finalizar la sesión de campo, revisa el resumen y puede iniciar la sincronización manualmente o dejar que se realice en segundo plano al recuperar cobertura de red.

7.6. Mecanismo de Sincronización Offline

7.6.1. Patrón Outbox Queue

El mecanismo de sincronización se diseña siguiendo el patrón **Outbox Queue** (cola de salida), ampliamente utilizado en sistemas distribuidos con conectividad intermitente. El funcionamiento es el siguiente:

1. Cuando el usuario crea o modifica un registro (sesión u observación) en el dispositivo sin conexión, el registro se persiste en SQLite con `sync_state = PENDING` y se inserta una entrada en la tabla `sync_queue` con el *payload* completo del cambio.
2. Cuando el dispositivo recupera conectividad, una tarea de sincronización en segundo plano o una acción manual del usuario lee la cola y realiza una petición `POST /api/sync/batch` con todos los registros pendientes.
3. El servidor procesa los registros en orden, los persiste en PostgreSQL y devuelve el estado canónico actualizado.
4. El cliente actualiza el `sync_state` de los registros procesados a `SYNCED` y descarga los cambios remotos realizados por otros usuarios mediante el endpoint `GET /api/sync/changes`, enviando `last_sync_timestamp` como parámetro de consulta.
5. En caso de error en un registro concreto (conflicto de datos, registro inválido), ese registro pasa a `sync_state = ERROR` y se notifica al usuario, sin bloquear la sincronización del resto.

7.6.2. Estados de sincronización

Cada sesión y observación puede encontrarse en uno de cuatro estados, como muestra la tabla 9.

Cuadro 9: Estados de sincronización y su significado.

Estado	Significado
PENDING	El registro existe solo en el dispositivo; aún no se ha intentado sincronizar.
SYNCING	El registro está siendo procesado en la petición de sincronización actual.
SYNCED	El servidor ha confirmado la recepción y persistencia del registro.
ERROR	El servidor rechazó el registro (conflicto, datos inválidos). Requiere intervención manual o reintento.

7.6.3. Resolución de conflictos

La estrategia de resolución de conflictos propuesta es **last-write-wins** con granularidad de registro: si dos usuarios modifican la misma sesión de forma independiente mientras están sin conexión, prevalece la versión con el **timestamp** más reciente. Esta estrategia es suficiente para el modelo de uso de OmniLitter, donde cada sesión de muestreo pertenece a un único observador responsable y los conflictos genuinos son infrecuentes.

7.6.4. Garantía de identificadores únicos en entorno offline

Para que la estrategia Outbox Queue funcione sin depender de un identificador asignado por el servidor en el momento de la captura, los registros creados en modo offline usan **UUID v4 generados en el cliente**. Esto garantiza que las sesiones y observaciones creadas simultáneamente en distintos dispositivos no colisionarán al sincronizarse, independientemente del orden de llegada al servidor.

Referencias

- [1] Meta Open Source. *React Documentation*. Disponible en: <https://react.dev> [Último acceso: mayo 2026].
- [2] Meta Open Source. *React Native Documentation*. Disponible en: <https://reactnative.dev> [Último acceso: mayo 2026].
- [3] Expo. *Expo Documentation*. Disponible en: <https://docs.expo.dev> [Último acceso: mayo 2026].
- [4] SQLite Consortium. *SQLite Documentation*. Disponible en: <https://www.sqlite.org/docs.html> [Último acceso: mayo 2026].
- [5] The PostgreSQL Global Development Group. *PostgreSQL Documentation*. Disponible en: <https://www.postgresql.org/docs/> [Último acceso: mayo 2026].
- [6] Prisma Data, Inc. *Prisma Documentation*. Disponible en: <https://www.prisma.io/docs> [Último acceso: mayo 2026].
- [7] OpenJS Foundation. *Express Documentation*. Disponible en: <https://expressjs.com> [Último acceso: mayo 2026].
- [8] Vite. *Vite Documentation*. Disponible en: <https://vite.dev> [Último acceso: mayo 2026].

A. Declaración del Uso de IA

En cumplimiento con las buenas prácticas académicas y de transparencia, se declara que para la elaboración de este trabajo se ha contado con el apoyo de herramientas de Inteligencia Artificial Generativa. El uso de dichas herramientas se ha limitado a tareas de apoyo, contraste y revisión, manteniendo los autores la responsabilidad final sobre las decisiones técnicas, el código entregado y la redacción definitiva de la memoria.

A.1. Usos realizados

Las herramientas de IA se han utilizado principalmente en las siguientes tareas:

- **Organización inicial del trabajo.** Apoyo para estructurar la memoria, ordenar capítulos y decidir qué artefactos documentales eran necesarios.
- **Contraste de stack tecnológico.** Comparación de alternativas para el portal web, la aplicación móvil, el backend, la base de datos y el mecanismo de sincronización offline.
- **Prototipado de interfaces.** Ayuda en la generación y ajuste de mockups funcionales del portal web y de la aplicación móvil.
- **Diseño de arquitectura.** Apoyo en la elaboración de diagramas, separación de componentes, flujo de sincronización y modelo de datos.
- **Revisión de texto.** Reformulación y mejora de redacción de apartados de la memoria, siempre sobre contenido revisado por los autores.

El Cuadro 10 resume las partes del trabajo donde se utilizó IA y con qué propósito.

Cuadro 10: Detalle del uso de IA en el trabajo.

Parte del trabajo	Herramienta	Propósito
Estructura de memoria	Claude / Codex	Proponer organización de capítulos y anexos.
Stack tecnológico	Claude / Codex	Comparar alternativas y razonar decisiones técnicas.
Mockups funcionales	Claude / Codex	Apoyar la generación y corrección de pantallas React.
Diseño del sistema	Claude / Codex	Redactar borradores, diagramas y tablas técnicas.
Revisión de redacción	Claude / Codex	Mejorar claridad, coherencia y estilo académico.

A.2. Ejemplos de prompts utilizados

A continuación se recogen ejemplos representativos de prompts empleados durante el proyecto. No se incluyen todas las conversaciones, sino aquellos tipos de consulta que reflejan el uso realizado.

Elección del stack

Estamos diseñando una aplicación móvil de campo que debe funcionar sin conexión, capturar GPS y fotos, y sincronizar observaciones con un backend cuando recupere conectividad. Compara alternativas para la base de datos local y razona si tiene sentido usar SQLite directamente en este contexto.

Diseño de arquitectura

Necesito plantear el capítulo de diseño de la solución para una plataforma formada por portal web, app móvil offline-first y backend REST. Propone una estructura para explicar arquitectura, componentes, base de datos y sincronización.

Revisión de requisitos

Tenemos requisitos obtenidos en reuniones con el cliente y los hemos contrastado mediante mockups. Ayúdame a organizarlos en requisitos funcionales, no funcionales y de datos científicos, manteniendo trazabilidad con las reuniones.

Revisión de redacción

He redactado un apartado de la memoria y quiero que suene más claro y técnicamente correcto. Reformula el texto manteniendo el contenido y las decisiones tomadas por el equipo.

A.3. Autoría y revisión crítica

Los autores declaran conocer y poder explicar el contenido incorporado al trabajo. Las respuestas obtenidas de las herramientas de IA se trataron como material de apoyo y fueron revisadas críticamente antes de su inclusión. La selección del dominio, la validación con el cliente, la toma de decisiones, la implementación final, la ejecución de pruebas y la entrega definitiva son responsabilidad de los autores.

B. Requisitos Detallados y Validación con Cliente

Este anexo recoge de forma más detallada la información utilizada para construir el registro de requisitos del proyecto y el proceso de validación seguido con el cliente. Su objetivo es dejar constancia del origen de las decisiones de diseño y de la evolución del alcance tras contrastar la propuesta inicial.

B.1. Resumen de reuniones con el cliente

Cuadro 11: Resumen de reuniones mantenidas con el cliente.

Reunión	Objetivo	Resultado principal
Toma inicial de requisitos	Comprender el problema, el alcance esperado y las limitaciones de la herramienta existente.	Se identificó la necesidad de una solución formada por aplicación móvil, portal web, funcionamiento offline, grupos de usuarios, mapas, dashboards y taxonomía de residuos.
Revisión de requisitos y mockups	Contrastar los requisitos definidos con los prototipos interactivos del portal y la aplicación móvil.	Se validó que los mockups cubrían los procesos principales y se reforzó la prioridad de la captura offline, la separación por grupos y la trazabilidad de los datos científicos.

B.2. Requisitos obtenidos en la reunión inicial

Durante la primera reunión se extrajeron los siguientes requisitos de alto nivel:

1. Desarrollar una plataforma compuesta por aplicación móvil y portal web.
2. Permitir registrar residuos en distintos entornos naturales, no solo ríos y mares, sino también playas, bosques y otros terrenos.
3. Registrar información científica mínima de cada observación: tipo de residuo, cantidad, posición, fecha, hora, entorno y comentarios.
4. Permitir adjuntar fotografías a las observaciones.
5. Garantizar que la aplicación móvil pueda utilizarse sin conexión a internet.
6. Sincronizar automáticamente los datos cuando el dispositivo recupere conectividad.
7. Gestionar usuarios, grupos y permisos para mantener la privacidad de los datos de cada equipo.

8. Ofrecer en el portal web herramientas de consulta, mapas, dashboards y exportación de datos.
9. Mantener una taxonomía de residuos jerárquica, navegable y ampliable.
10. Preservar trazabilidad científica mediante versiones de taxonomía, metadatos de sesión y registros de auditoría.

B.3. Validación de requisitos mediante mockups

La segunda reunión se centró en revisar los prototipos funcionales y comprobar que los flujos implementados representaban correctamente los requisitos ya recogidos. En lugar de generar un nuevo bloque de requisitos, esta revisión permitió confirmar la cobertura de los casos de uso principales:

- Inicio de sesión y diferenciación entre perfiles de administrador y colaborador.
- Selección de grupo activo y filtrado de la información asociada.
- Creación de sesiones de muestreo en la aplicación móvil.
- Registro de observaciones con taxonomía, cantidad, tamaño, color, comentarios, GPS y fotografía.
- Visualización de observaciones y sesiones desde el portal web.
- Consulta geográfica mediante mapas.
- Representación del estado de sincronización y de los datos pendientes.

Los ajustes visuales e internos realizados durante la construcción de los mockups se consideran decisiones de diseño del equipo y no requisitos adicionales solicitados por el cliente. Por ello no se documentan como cambios de alcance, sino como parte del proceso de refinamiento del prototipo.

B.4. Requisitos funcionales detallados

El registro de requisitos funcionales queda agrupado en seis bloques:

- **Autenticación y sesión:** inicio de sesión, cierre de sesión y recuperación de contraseña.
- **Roles, grupos y permisos:** gestión de grupos, usuarios, membresías y selección de grupo activo.
- **Captura de campo:** sesiones de muestreo, observaciones, GPS, fotografías, cantidad, tamaño, color y comentarios.
- **Taxonomía:** navegación jerárquica, búsqueda y favoritos.

- **Sincronización:** funcionamiento offline, cola de sincronización, sincronización manual y automática, y estados por elemento.
- **Portal web:** listados, filtros, mapas, heatmaps, dashboards, fotografías, exportación, grupos, usuarios y configuración.

B.5. Requisitos no funcionales detallados

Los requisitos no funcionales se agrupan en:

- **Seguridad:** comunicación cifrada, control de acceso por grupo y protección de adjuntos.
- **Fiabilidad:** sincronización reintentable y preservación de datos ante fallos o cierres inesperados.
- **Usabilidad:** interfaz adaptada a uso de campo en móvil y tablet.
- **Operación:** logs, copias de seguridad y matriz requisito-prueba.
- **Compatibilidad:** soporte para dispositivos móviles modernos, con especial atención a Android e iOS.

B.6. Requisitos de datos científicos

Los requisitos de datos científicos se centran en garantizar que las observaciones registradas puedan analizarse posteriormente sin perder contexto:

- Taxonomía jerárquica con códigos estables.
- Metadatos de muestreo por sesión.
- Metadatos de foto cuando estén disponibles.
- Extensión versionada de la taxonomía.
- Validaciones de obligatoriedad, rango y consistencia.
- Exportaciones con diccionario de columnas y unidades.
- Versión de taxonomía y de aplicación asociada a sesiones/exportaciones.